DSA 495 601 Project Milestone 6

Thomas Grover

- Introduction
- Setup
- 1. Data consistency checks
  - 1.1 Feature availability check
  - 1.2 Categorical variable consistency
  - 1.3 Distribution comparison
- 2. Apply model to test set
- 3. Compare predicted probability distributions
  - 3.1 Overall distribution comparison
  - 3.2 Distribution by true outcome
- 4. Discrimination performance
  - 4.1 ROC curve comparison
  - 4.2 Precision-Recall curve comparison
- 5. Classification performance at selected thresholds
  - 5.1 Visualize classification performance
- 6. Fairness assessment on test set
  - 6.1 Distribution by demographic groups
  - 6.2 Classification metrics by group
  - If we deployed the model with these biases, the disparities may cause us to over target certain groups (higher FPR) and under target other groups (higher FNR), which could lead to unequal access to opportunity. There are some important ethical considerations with this when it comes to banking/marketing. It isn't a huge issue because the disparities are consistent across training/testing, but this is something that I would recommend is monitored over time if the model is deployed.
- 7. Final recommendations
- 8. Save final results

Introduction

For Milestone 6, you will test your selected learner from Milestone 5 on your held-out test set. This milestone represents the culmination of your predictive modeling project, where you evaluate how well your model generalizes to completely unseen data.

Your goals:

1. Apply your selected learner to the test set
2. Compare test performance to validation performance
3. Evaluate discrimination and calibration on test data
4. Assess fairness metrics on the test population
5. Provide final recommendations for model deployment

Important notes:

- The test set should only be used once - this is your final evaluation
- All coding you will need to modify or expand are marked with @TODO
- Answer all questions marked with @TODO

After completing all sections, knit to HTML, then download and print/save as PDF using this naming convention: `DSC495_601_FA25_Milestone6_unityID.pdf`

Due date: November 13 11:45 a.m. ET

Setup

This section loads all necessary files and configures your R environment for model testing.

```r
# Set random seed for reproducibility
seed <- 2024

# Configure notebook settings
knitr::opts_chunk$set(
  cache = FALSE,      # Don't cache results
  warning = FALSE,    # Suppress warnings
  message = FALSE,    # Suppress messages
  fig.width = 8,
  fig.height = 7
)

# Load helper scripts with custom functions
# These must be in your R/ subdirectory
source("/share/dsa495601f25/templates/R/packages.R")
source("/share/dsa495601f25/templates/R/learner_training.R")
source("/share/dsa495601f25/templates/R/learner_training_helpers.R")
source("/share/dsa495601f25/templates/R/equity_metrics.R")
source("/share/dsa495601f25/templates/R/performance_metrics.R")
source("/share/dsa495601f25/templates/R/plots_updated.R")

# Load your processed data from Milestone 3
trainDat <- readRDS(paste0(params$my_path, "trainDat.rds"))
testDat  <- readRDS(paste0(params$my_path, "testDat.rds"))

# Load learner specifications from Milestone 4
learnerSpec <- readRDS(paste0(params$my_path, "learnerSpec.rds"))

# Load preprocessing recipes from Milestone 4
recipes <- readRDS(paste0(params$my_path, "recipes.rds"))
mainRecipes <- readRDS(paste0(params$my_path, "recipe_main.rds"))

# Load validation results from Milestone 5
trainResults <- readRDS(paste0(params$my_path, "validateLearnerResults.rds"))

# Extract key components
trainMetrics <- trainResults$learnersMetrics
trainPredProbs <- trainResults$predProbs


# Verify everything loaded correctly
cat("\nData verification:\n")
```

```
##
## Data verification:
```

```r
cat("Training data:", nrow(trainDat), "rows,", ncol(trainDat), "columns\n")
```

```
## Training data: 31647 rows, 73 columns
```

```
cat("Testing data:", nrow(testDat), "rows,", ncol(testDat), "columns\n")
```

```
## Testing data: 13564 rows, 73 columns
```

```
cat("Selected learner:", trainResults$modelResults$learnerName[[1]], "\n")
```

```
## Selected learner: random_forest_predSet_kitchensink
```

```
cat("Selected threshold(s):", trainResults$selectedThreshold, "\n")
```

```
## Selected threshold(s): 0.1 0.15
```

```
cat("Outcome variable:", learnerSpec$outcomeName, "\n")
```

```
## Outcome variable: y_binary
```

---

1. Data consistency checks

Before applying your model to the test set, we need to verify that the test data is compatible with your trained model.

1.1 Feature availability check

Verify that all predictors used in training are present in the test set.

```
# Extract predictors used in the selected model
outcome <- learnerSpec$outcomeName
learnerName <- trainResults$modelResults$learnerName[[1]]
model_vars <- colnames(trainResults$modelFits[[learnerName]]$pre$mold$predictors)

# Check for missing predictors in test set
missing_in_test <- setdiff(model_vars, names(testDat))

# Display results
if (length(missing_in_test) == 0) {
  cat("✓ All required predictors are present in the test dataset.\n")
} else {
  cat("⚠ WARNING: The following predictors are missing from the test set:\n")
  cat(paste("-", missing_in_test, collapse = "\n"), "\n")
}
```

```
## ✓ All required predictors are present in the test dataset.
```

```
cat("\nNumber of predictors in model:", length(model_vars), "\n")
```

```
##
## Number of predictors in model: 44
```

Question 1: If any predictors are missing from the test set, what might have caused this? How would missing predictors affect your model's ability to make predictions? What steps would you take to address this issue?

In my case, the test set has all the required predictors necessary to apply my model. If there were somehow predictors missing, it could be caused by issues when creating the datasets, such as creating a dummy variable in one set but not the other, or when saving the datasets. Having an incomplete test set could cause invalid predictions because the model would be trying to extrapolate predictive info from an incomplete feature set.

This is an important part of the predictive analytics pipeline, and this is the reason we had such attention to detail in the data processing steps of the project. It is essential that training and testing sets are essentially identical, and this is why we focus so much on an identical data processing pipeline when creating and saving the two datasets.

1.2 Categorical variable consistency

Check whether categorical variables have consistent levels between training and test sets.

```r
# Extract categorical variables
all_categorical_vars <- names(trainDat %>%
                              select(where(~ is.character(.) | is.factor(.))))

# Only check categorical variables that are in the model
model_categorical_vars <- intersect(all_categorical_vars, model_vars)

if (length(model_categorical_vars) > 0) {
  # Compare levels for each categorical variable
  category_differences <- map_dfr(model_categorical_vars, function(var) {
    train_levels <- unique(trainDat[[var]])
    test_levels <- unique(testDat[[var]])

    new_in_test <- setdiff(test_levels, train_levels)
    missing_in_test <- setdiff(train_levels, test_levels)

    if (length(new_in_test) > 0 | length(missing_in_test) > 0) {
      tibble(
        Variable = var,
        New_in_test = ifelse(length(new_in_test) > 0,
                             paste(new_in_test, collapse = ", "),
                             "None"),
        Missing_in_test = ifelse(length(missing_in_test) > 0,
                                 paste(missing_in_test, collapse = ", "),
                                 "None")
      )
    } else {
      NULL
    }
  })

  if (nrow(category_differences) > 0) {
    cat("⚠ Categorical variable differences detected:\n")
    print(category_differences)
  } else {
    cat("✓ All categorical variables have consistent levels.\n")
  }
} else {
  cat("No categorical variables in the model.\n")
}
```

```
## No categorical variables in the model.
```

Question 2: If there are new categorical levels in the test set that weren't in the training set, how might your model handle these? What preprocessing steps in your recipe (from Milestone 4) might help address this issue? Consider the implications for model deployment in production.

In this case, there are no new categorical variables because I one-hot encoded them all during the pre-processing phase. If there were new categorical variables, the model would not know how to handle them. Again, this is why we put so much emphasis on our data pre-processing, and creating identical training and testing sets. This shows the importance of ensuring that the data we are using for training/testing has the same categories.

1.3 Distribution comparison

Compare the distributions of predictors between training and test sets.

```
key_predictors <- c("age_std", "balance_std", "high_balance", "long_call")

# Filter to only predictors that exist in both datasets
key_predictors <- intersect(key_predictors, model_vars)

if (length(key_predictors) > 0) {
  # Create comparison summaries
  cat("=== Distribution Comparison for Key Predictors ===\n\n")

  for (pred in key_predictors) {
    cat("Variable:", pred, "\n")

    if (is.numeric(trainDat[[pred]])) {
      # Numeric variable comparison
      print(summary(trainDat[[pred]]))
      print(summary(testDat[[pred]]))

    } else {
      # Categorical variable comparison
      train_table <- table(trainDat[[pred]]) / nrow(trainDat)
      test_table <- table(testDat[[pred]]) / nrow(testDat)

      cat("  Training proportions:\n")
      print(round(train_table, 3))
      cat("  Test proportions:\n")
      print(round(test_table, 3))
    }
    cat("\n")
  }
} else {
  cat("No key predictors found in model variables.\n")
}
```

```
## === Distribution Comparison for Key Predictors ===
##
## Variable: age_std
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -2.1602 -0.7491 -0.1847  0.0000  0.6619  5.0831
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -2.1595 -0.8377 -0.1767  0.0000  0.6730  5.0162
##
## Variable: balance_std
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -2.74207 -0.42869 -0.30197  0.00000  0.02436 33.69358
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -2.96818 -0.41340 -0.29657  0.00000  0.01485 30.63967
##
## Variable: high_balance
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.0     0.0     0.0     0.2     0.0     1.0
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  0.0000  0.0000  0.2005  0.0000  1.0000
##
## Variable: long_call
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  0.0000  0.0000  0.2999  1.0000  1.0000
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0000  0.0000  0.0000  0.3046  1.0000  1.0000
```

Question 3: Examine the distribution comparisons above. Do you observe any substantial differences between training and test sets? If distributions differ notably, what might this suggest about how your data was split or about temporal changes in your population? How might these differences impact your model's test performance?

Across the 4 key predictors, the two sets are extremely similar in distribution. This suggests that we were quite successful in creating a training and testing set that are comparable. Becuase the two sets are so similar, we expect that the model will generalize to new/unseen data well. If there were significant differences between the two sets, we could expect a greater deviation between training and testing results, with test results being weaker. However, since the two sets are so similar in this case, we do not have to worry about that.

---

2. Apply model to test set

Now we'll apply your selected learner to the test set and generate predictions.

```
# Apply model to test set
testResults <- test_learners(
  trainResults = trainResults,
  learnerSpec = learnerSpec,
  mainRecipes = mainRecipes,
  trn_data = trainDat,
  tst_data = testDat
)

# Extract results
testMetrics <- testResults$testMetrics
testPredProbs <- testResults$testPredProbs
testRocResults <- testResults$testRocResults
testPrResults <- testResults$testPrResults
testFits <- testResults$testFits

# Combine predictions for comparison
testPredProbs$set <- "Test"
trainPredProbs$set <- "Validation"
trainPredProbs[[learnerSpec$outcomeName]] <- as.factor(trainPredProbs[[learnerSpec$outcomeNam
e]])
testPredProbs[[learnerSpec$outcomeName]] <- as.factor(testPredProbs[[learnerSpec$outcomeName]])
allPredProbs <- bind_rows(trainPredProbs, testPredProbs)

cat("\n=== Test Set Predictions Summary ===\n")
```

```
##
## === Test Set Predictions Summary ===
```

```
cat("Number of test observations:", nrow(testPredProbs), "\n")
```

```
## Number of test observations: 13564
```

```
cat("Predicted probability range:",
    round(min(testPredProbs$.pred_yes), 3), "to",
    round(max(testPredProbs$.pred_yes), 3), "\n")
```

```
## Predicted probability range: 0 to 0.857
```

---

3. Compare predicted probability distributions

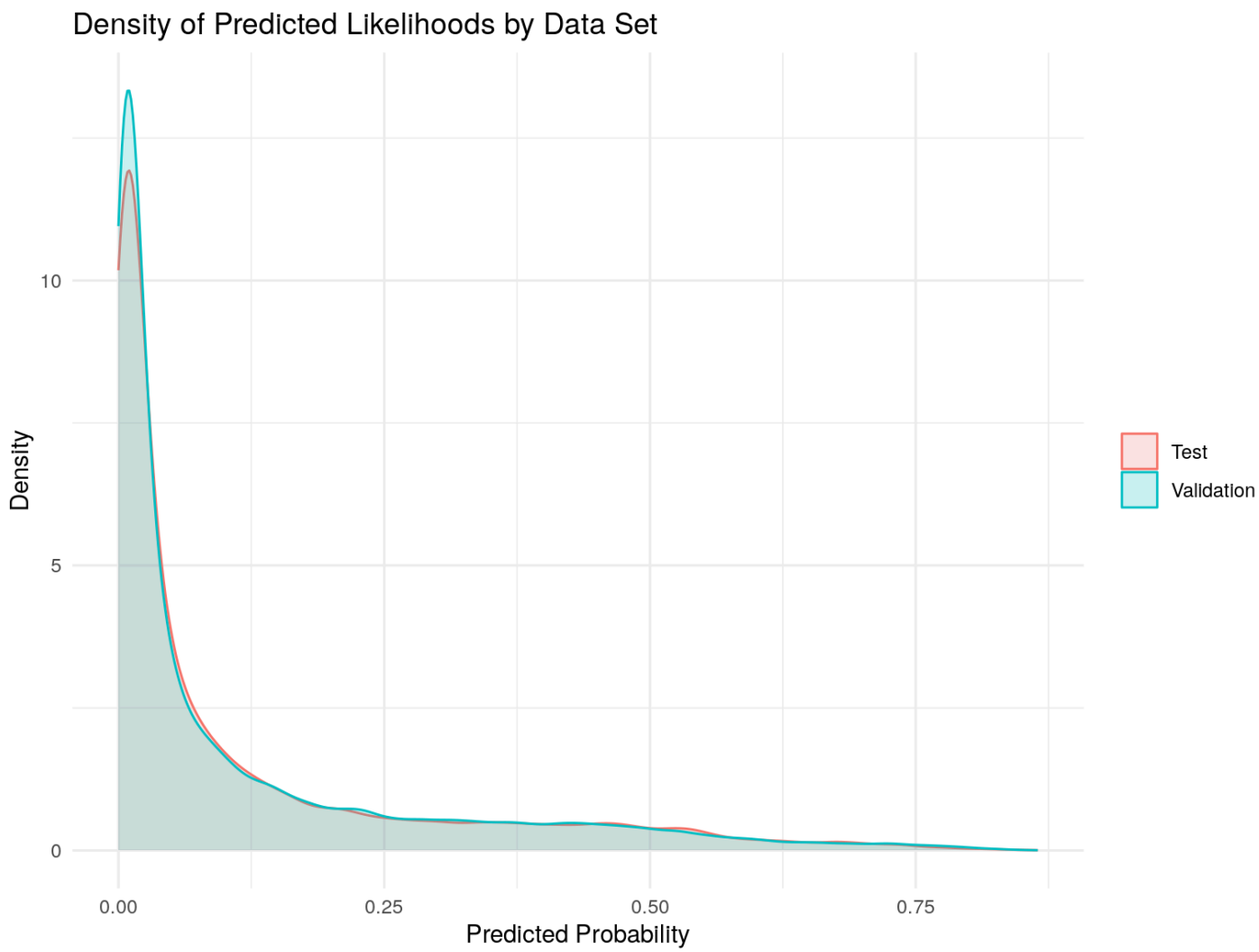
Understanding how predicted probabilities differ between validation and test sets helps identify potential overfitting or data drift.

3.1 Overall distribution comparison

This plot compares the distribution of predicted probabilities between your validation set (from cross-validation) and the test set.

What to look for: - Similar distributions suggest good generalization - Shifted distributions may indicate overfitting or population differences - Narrower test distribution might suggest the model is less confident on new data

```
# Plot comparison of predicted probability distributions
plot_predicted_probabilities_bySet(allPredProbs)
```



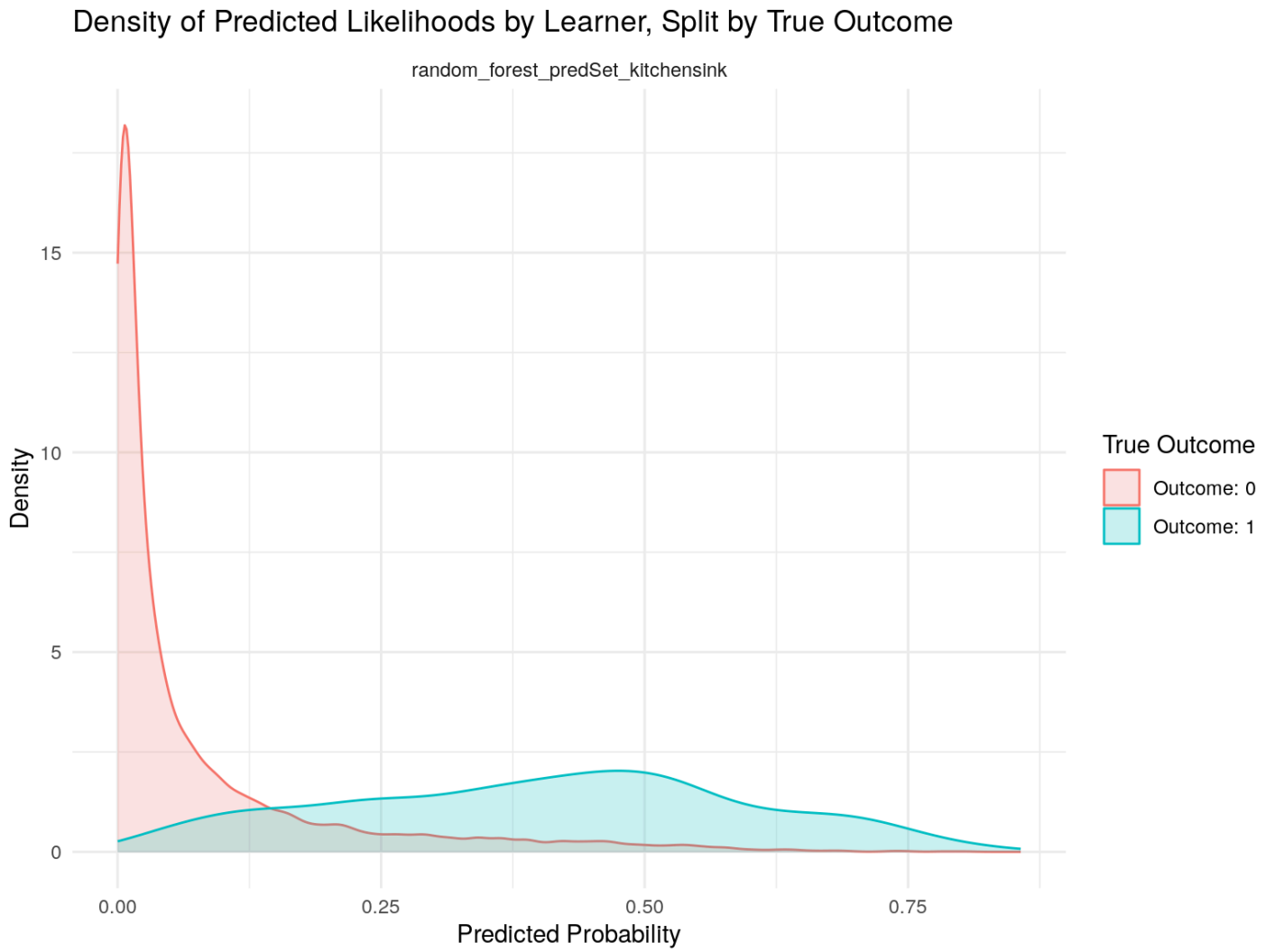
Question 4: Compare the validation and test distributions. Are they similar in shape and range? If the test distribution is shifted (e.g., toward lower probabilities), what might this indicate about your model's generalization? Consider both overfitting and potential differences in the test population.

The two distributions are extremely similar in shape and range. This shows that our chosen model generalizes well. If the test distrubution was shifted toward lower probabilities, this could indicate that our model is conservative when predicting on unseen data. The extreme similarity in this case however shows that the training and testing populations were very similar.

3.2 Distribution by true outcome

Examine how well the model separates positive and negative cases in the test set.

```
# Plot test set predictions split by actual outcome
testPredProbs[[learnerSpec$outcomeName]] <- ifelse(testPredProbs[[learnerSpec$outcomeName]]==
1,"yes","no")
plot_predicted_probabilities_split(testPredProbs, learnerSpec$outcomeName)
```



Question 5: In the test set, how well does your model separate the predicted probabilities for true positives (outcome=1) versus true negatives (outcome=0)? Is the separation comparable to what you observed in validation (Milestone 5)? If separation is worse in the test set, what might explain this degradation?

In the test set, the model continues to show clear separation between outcomes. With negatives, the model predictions are concentrated close to zero, and with positives, the models predictions have a broader distribution, centered around about 0.5. Overall, the model still shows strong separation, and very little degradation compared to the training set.

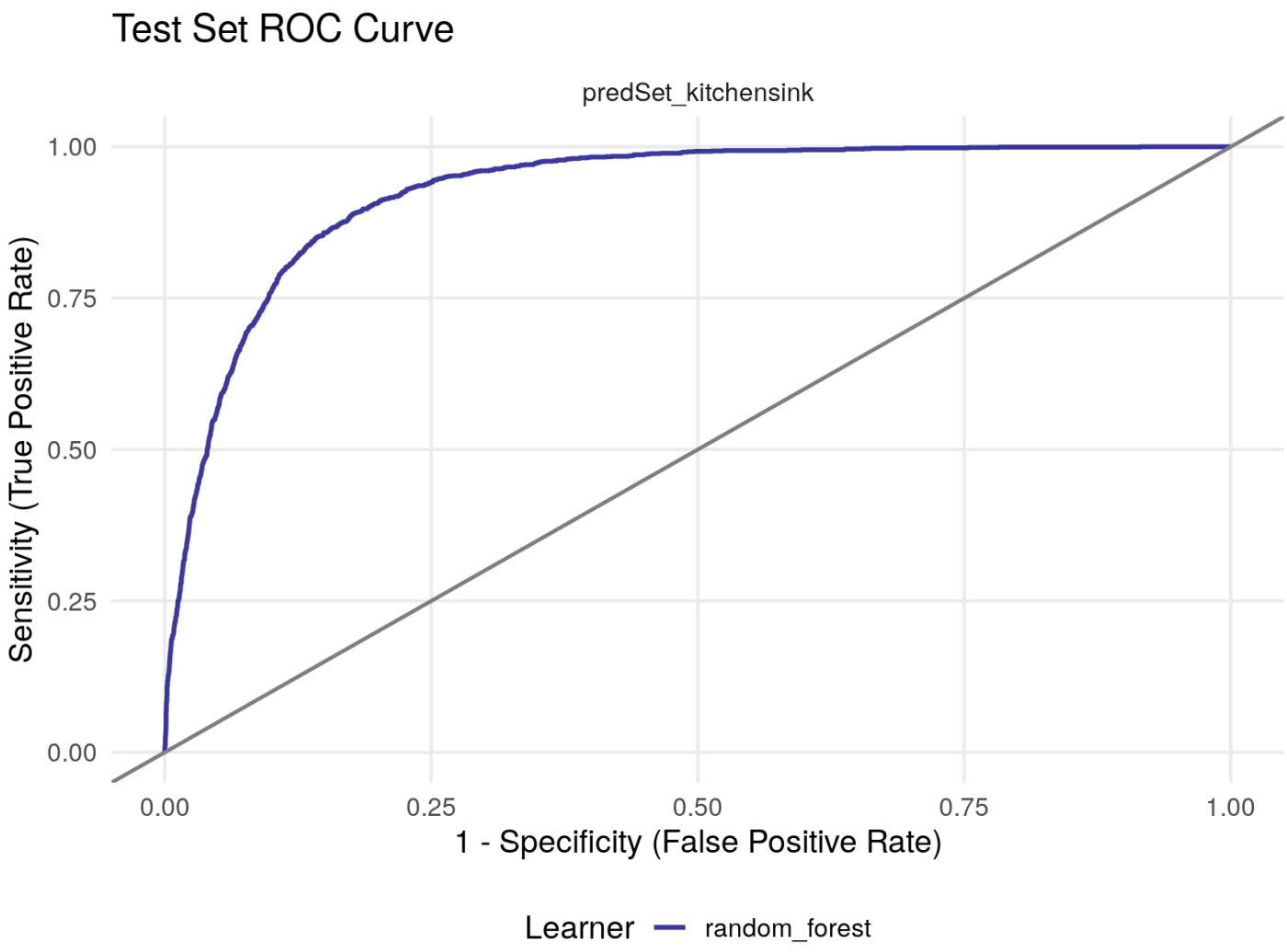
---

4. Discrimination performance

Evaluate how well your model distinguishes between classes in the test set.

4.1 ROC curve comparison

```
# Plot ROC curve for test set
plot_roc_curve_with_reference(
  testRocResults,
  referencePredSet = unique(testRocResults$predSet),
  title = "Test Set ROC Curve"
)
```




```
# Display ROC-AUC comparison
cat("\n=== ROC-AUC Comparison ===\n")
```

```
##
## === ROC-AUC Comparison ===
```

```
cat("Validation ROC-AUC:",
    round(trainResults$modelResults %>%
          filter(.metric == "roc_auc") %>%
          pull(mean), 3), "\n")
```

```
## Validation ROC-AUC: 0.922
```

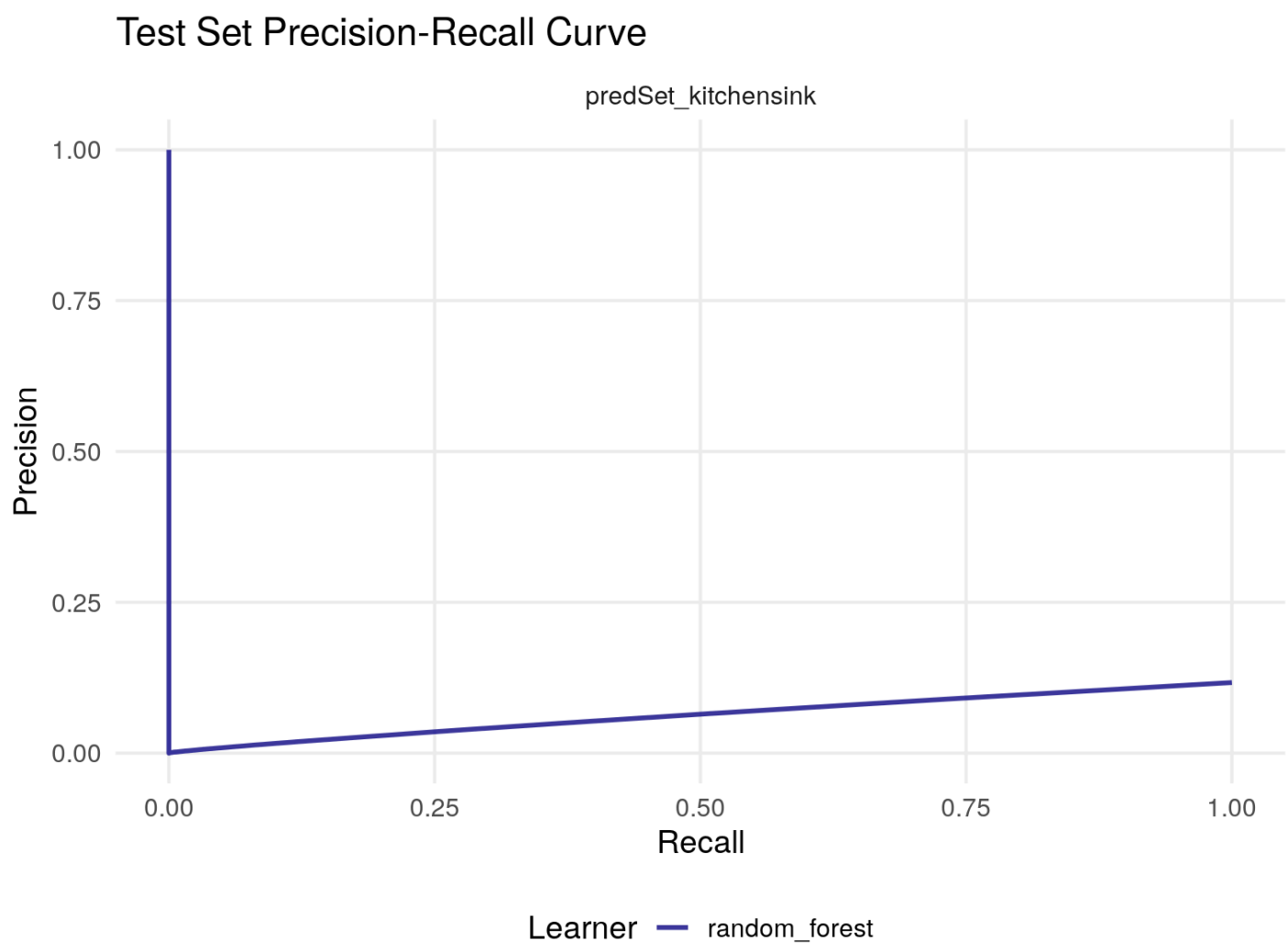
```
cat("Test ROC-AUC:",
    round(testMetrics$roc_auc, 3), "\n")
```

```
## Test ROC-AUC: 0.926
```

4.2 Precision-Recall curve comparison

For imbalanced outcomes, PR curves provide more informative performance assessment.

```
# Plot PR curve for test set
plot_pr_curve_with_reference(
  testPrResults,
  referencePredSet = unique(testPrResults$predSet),
  title = "Test Set Precision-Recall Curve"
)
```



```
# Display PR-AUC comparison
cat("\n=== PR-AUC Comparison ===\n")
```

```
##
## === PR-AUC Comparison ===
```

```
cat("Validation PR-AUC:",
    round(trainResults$modelResults %>%
          filter(.metric == "pr_auc") %>%
          pull(mean), 3), "\n")
```

```
## Validation PR-AUC: 0.592
```

```
cat("Test PR-AUC:",
    round(testMetrics$pr_auc, 3), "\n")
```

```
## Test PR-AUC: 0.616
```

Question 6: Compare the discrimination metrics (ROC-AUC and PR-AUC) between validation and test sets. Is the performance drop within acceptable ranges (typically <5-10% for well-generalizing models)? Given your outcome prevalence, which metric (ROC-AUC or PR-AUC) provides a more reliable assessment of performance degradation? If you observe a large drop, what might be the causes?

Both discrimination metrics actually increased when going to the test set, indicating that the model generalizes quite well. Given the imbalance in our target variable, PR-AUC is the more reliable assessment of performance. Seeing no drop whatsoever, and actually an increase in this metric, indicates that our model is generalizing well and may have actually learned predictice patterns in training. A common reason for large drops in performance is population drift, but that is not the case here, and our model generalizes well to the unseen testing set.

---

5. Classification performance at selected thresholds

Evaluate classification metrics using the threshold(s) you selected in Milestone 5.

```r
# Define metric groups
rateMetrics <- c("tpr", "fpr", "tnr", "fnr")
countMetrics <- c("tpc", "fpc", "tnc", "fnc")
otherMetrics <- c("ppv", "f1", "acc")
allMetrics <- c(rateMetrics, countMetrics, otherMetrics)

# # To simplify, let's pick just one threshold and specify here
selectedThreshold <- 0.1

# Calculate test set metrics at selected thresholds
testMetricsThres <- conf_metrics_thres_learner(
  predProbs = testPredProbs,
  outcomeName = outcome,
  thresSeq = selectedThreshold
)

# Display test metrics
cat("=== Test Set Classification Metrics ===\n\n")
```

```
## === Test Set Classification Metrics ===
```

```r
testMetricsThres %>%
  select(threshold, all_of(c("tpr", "fpr", "ppv", "f1", "acc"))) %>%
  mutate(across(where(is.numeric), ~round(., 3))) %>%
  print()
```

```
##   threshold   tpr   fpr   ppv    f1   acc
## 1       0.1 0.934 0.235 0.345 0.504 0.785
```

```r
# Compare with validation metrics
  cat("\n=== Validation vs Test Comparison ===\n")
```

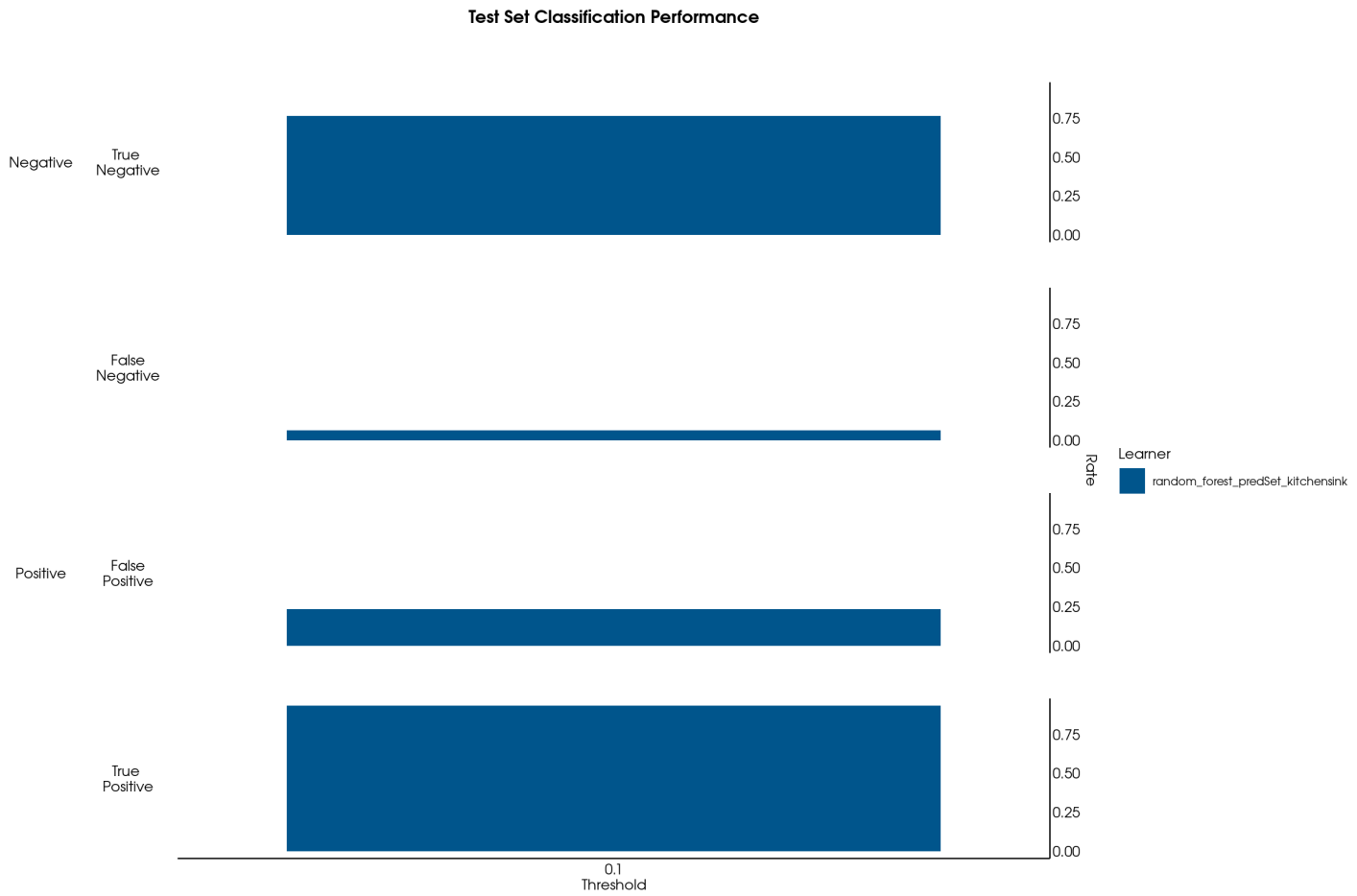
```
##
## === Validation vs Test Comparison ===
```

```
  # Get validation metrics at same threshold
  validationMetricsThres <- conf_metrics_thres_learner(
    predProbs = trainPredProbs,
    outcomeName = outcome,
    thresSeq = selectedThreshold
  )

  comparison <- data.frame(
    Metric = c("TPR", "FPR", "Precision", "F1"),
    Validation = c(
      validationMetricsThres$tpr[1],
      validationMetricsThres$fpr[1],
      validationMetricsThres$ppv[1],
      validationMetricsThres$f1[1]
    ),
    Test = c(
      testMetricsThres$tpr[1],
      testMetricsThres$fpr[1],
      testMetricsThres$ppv[1],
      testMetricsThres$f1[1]
    )
  )
  comparison$Difference <- comparison$Test - comparison$Validation
  print(comparison)
```

```
##      Metric Validation      Test   Difference
## 1       TPR  0.9276067 0.9344675  0.006860850
## 2       FPR  0.2384326 0.2351173 -0.003315327
## 3 Precision  0.3401010 0.3449639  0.004862925
## 4        F1  0.4977172 0.5039076  0.006190351
```

5.1 Visualize classification performance

```
# Filter and reshape data for plotting
plotData <- testMetricsThres %>%
  filter(threshold %in% selectedThreshold) %>%
  pivot_longer(
    cols = all_of(allMetrics),
    names_to = "metricType",
    values_to = "value"
  )

# Create bar plot for test set performance
conf_mat_bar_plot(
  plotData = plotData %>% filter(metricType %in% rateMetrics),
  metricType = "rate",
  outcomeDescription = c("Negative", "Positive"),
  graphTitle = "Test Set Classification Performance"
)
```



Question 7: Review the classification metrics at your selected threshold(s). How do key metrics (TPR, FPR, Precision) compare between validation and test sets? Are the differences uniform across metrics, or do some degrade more than others? Based on these test results, would you recommend adjusting the threshold for deployment? Why or why not?

Key metrics are extremely similar between validation and test sets. True positive rate increased slightly when going to the test set, which perfectly aligns with my goal of minimizing false negatives. Precision also slightly increased when going to the test data. These differences are uniform, and make sense with the slight decrease in false positive rate. But in general, the metrics are extremely similar across the training and testing sets, which suggests minimal performance degradation.

Considering how stable these metrics our, and how in our bank marketing context minimizing false negatives is essential, there is really no reason to adjust the threshold for deployment. I think the current threshold of 0.1 is appropriate for this use case, and allows a nice balance between minimizing false negatives and false positives.

---

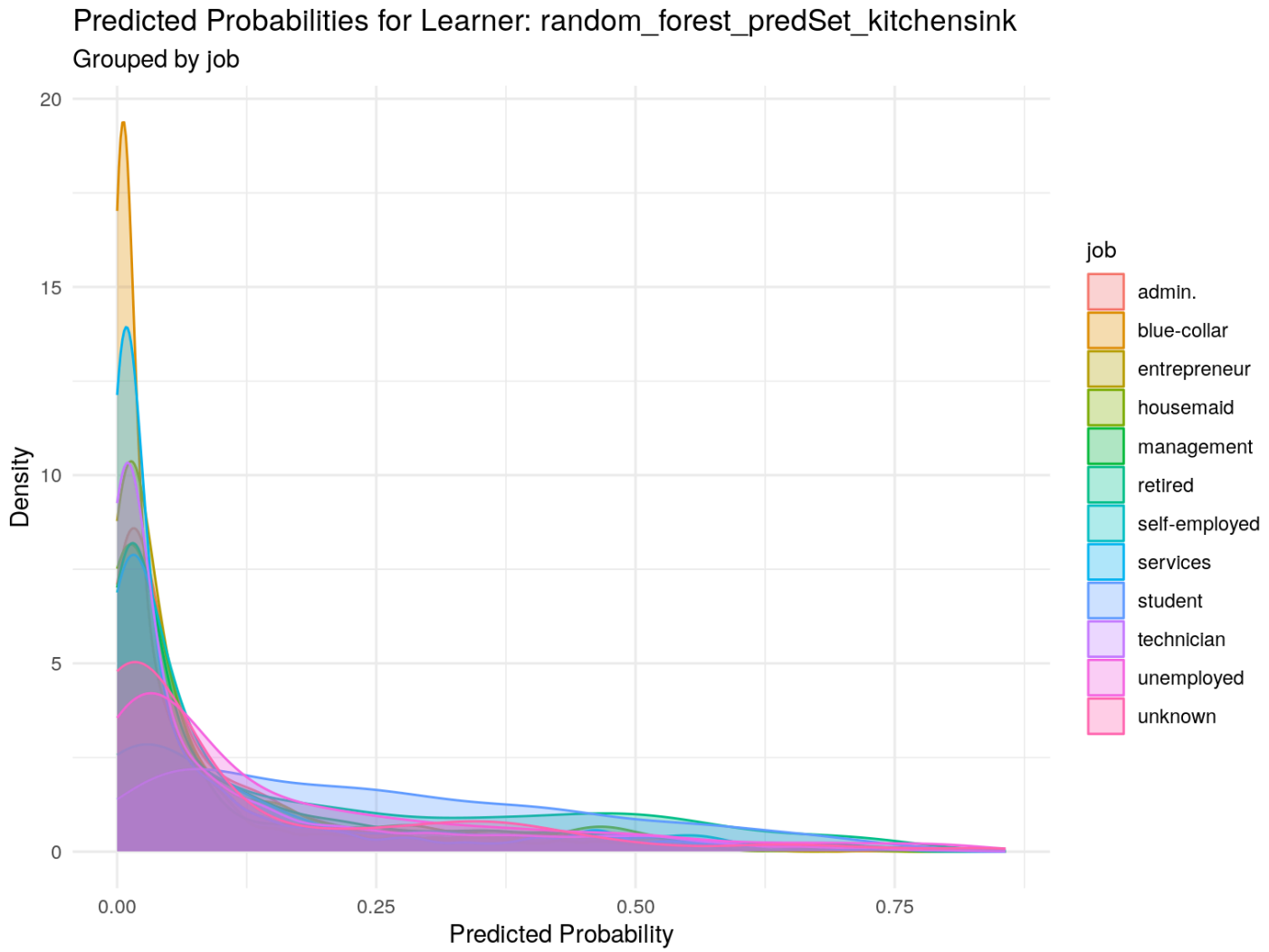
6. Fairness assessment on test set

Evaluate whether fairness issues observed in validation persist or change in the test set.

6.1 Distribution by demographic groups

```
equityVar <- learnerSpec$equityVars[1]  # Use first equity variable

# Check if equity variable exists in test set
if (equityVar %in% names(testDat)) {
  # Add equity variable to predictions if not already present
  if (!(equityVar %in% names(testPredProbs))) {
    testPredProbs <- testPredProbs %>%
      bind_cols(testDat %>% select(all_of(equityVar)))
  }

  cat("Assessing fairness for equity variable:", equityVar, "\n\n")

  # Plot distribution by group
  plot_pred_probs(
    testPredProbs,
    learner = unique(testPredProbs$learnerName),
    group_var = equityVar
  )
} else {
  cat("⚠ Equity variable", equityVar, "not found in test set.\n")
}
```

```
## Assessing fairness for equity variable: job
```



Question 8: Compare the distribution of predicted probabilities across demographic groups in the test set. Are the patterns similar to what you observed in validation? Have disparities increased, decreased, or remained stable? What might explain any changes you observe?

The patterns are very similar to what I observed in validation. Most groups are clustered close to 0, which makes sense given the low proportion of positive outcomes (about 12%). There are very few disparities between training and testing, and the majority of groups have remained quite stable. Differences could be explained by differences in training and testing populations, but again that is not the case here. We have very similar results across the 2 sets, which suggests that any variability accross groups is genuine, and not a fairness issue introduced in testing.

6.2 Classification metrics by group

```
if (equityVar %in% names(testPredProbs)) {
  # Calculate equity metrics
  equityResults <- calc_equity_metrics_all(
    predProbs = testPredProbs,
    learnerNames = unique(testPredProbs$learnerName),
    thresholdsSelect = selectedThreshold,  # Use primary threshold
    equityVars = equityVar,
    outcomeName = outcome
  )

  # Display results
  cat("=== Test Set Fairness Metrics ===\n")
  cat("Threshold:", selectedThreshold, "\n\n")

  equityResults %>%
    select(category, tpr, fpr, ppv, fnr) %>%
    mutate(across(where(is.numeric), ~round(., 3))) %>%
    print()

  # Statistical tests for disparities
  cat("\n=== Statistical Tests for Disparities ===\n")

  equityTests <- calc_equity_tests_all(
    equityResults,
    unique(testPredProbs$learnerName),
    equityVar,
    selectedThreshold
  )

  equityTests %>%
    select(-learnerName, -threshold) %>%
    mutate(across(where(is.numeric), ~round(., 3))) %>%
    print()

  cat("\nNote: p-values < 0.05 indicate statistically significant differences between groups\n")
}
```

```
## === Test Set Fairness Metrics ===
## Threshold: 0.1
##
##         category   tpr   fpr   ppv   fnr
## 1         admin. 0.914 0.253 0.331 0.086
## 2    blue-collar 0.885 0.148 0.327 0.115
## 3   entrepreneur 0.974 0.196 0.311 0.026
## 4      housemaid 0.933 0.217 0.298 0.067
## 5     management 0.962 0.277 0.322 0.038
## 6        retired 1.000 0.472 0.400 0.000
## 7  self-employed 0.981 0.218 0.384 0.019
## 8       services 0.881 0.175 0.335 0.119
## 9        student 0.967 0.616 0.416 0.033
## 10    technician 0.912 0.220 0.359 0.088
## 11    unemployed 0.938 0.348 0.347 0.062
## 12       unknown 0.900 0.250 0.300 0.100
##
## === Statistical Tests for Disparities ===
```

```
## # A tibble: 1 × 5
## # Groups:   learnerName, equityVar [1]
##   learnerName                       equityVar fnr_pVal fpr_pVal fdr_pVal
##   <chr>                             <chr>        <dbl>    <dbl>    <dbl>
## 1 random_forest_predSet_kitchensink job              0        0    0.105
##
## Note: p-values < 0.05 indicate statistically significant differences between groups
```

Question 9: Examine the fairness metrics and statistical tests. Which metrics show statistically significant disparities between groups? Are these disparities consistent with what you found in validation, or have they changed? Consider the practical implications: if deployed, how would these disparities affect different groups in your population? What mitigation strategies would you recommend?

Based on the fairness metrics and statistical inference, there are several groups with statistically significant differences. Across the job variable, there are significant differences in FNR and FPR, which indicates that the model behaves differently for the different groups of the job variable. However, these differences are consistent with the differences noticed in validation. Since these same biases exist in the test set, and we have seen how similar the training and testing sets are, this suggests that these biases are structural and not an accident or mistake.

If we deployed the model with these biases, the disparities may cause us to over target certain groups (higher FPR) and under target other groups (higher FNR), which could lead to unequal access to opportunity. There are some important ethical considerations with this when it comes to

banking/marketing. It isn't a huge issue because the disparities are consistent across training/testing, but this is something that I would recommend is monitored over time if the model is deployed.

7. Final recommendations

Based on your complete analysis across training, validation, and testing, provide your final recommendations.

Question 10: Overall Performance Assessment: Based on the test set results, does your model perform well enough for deployment in your intended use case? Consider the absolute performance metrics (not just relative to validation), the costs of different types of errors in your context, and whether the model meets minimum performance requirements you would set. What specific performance metrics support or contradict deployment?

Based on the results, I think the model does perform well enough for deployment in my intended use case. We achieved high values for both ROC-AUC and PR-AUC, which indicate that our model is solid at discrimination. We need to take the high ROC-AUC with a grain of salt because of the imbalance in our target, but the high PR-AUC and ROC-AUC still indicate strong discrimination. Additionally, the model does very well in terms of TPR, and adjacently FNR. This is super important in this business context, so getting strong values for these metrics here provides evidence for deployment. Precision is somewhat low in this case, which provides evidence against deployment. However, based on the context of marketing, outreach is not very costly. False positives are far less harmful in this case than false negatives, so I accept the lower precision metric. Overall, the model fits the performance requirements I would set.

Question 11: Generalization and Robustness: Evaluate how well your model generalized from validation to test. Was the performance drop acceptable? What does this suggest about the model's likely performance in real-world deployment? Are there specific conditions or populations where you would expect the model to perform better or worse based on your test results?

The model generalized extremely well from validation to test. The acceptable performance drop range is 5-10 percent, and with my model I actually saw a performance increase. Both the PR-AUC and ROC-AUC metrics increased marginally, which show that the model performs consistently with unseen data. The classification metrics were all extremely stable, again suggesting strong generalization to new data. The consistency between the two sets suggests that the model would likely perform reliably if deployed in a real-world context. There are some concerns about the model performing differently across groups of the job variable based on the fairness assessment, where the model may perform better/worse across certain values of the job variable. However, this is a minor concern with all of the other evidence suggesting strong generalization.

Question 12: Fairness and Ethical Considerations: Based on the fairness assessment in both validation and test sets, are there ethical concerns about deploying this model? Which groups might be advantaged or disadvantaged? What specific fairness interventions or monitoring would you recommend if this model were to be deployed? Consider both statistical fairness and practical impacts on affected populations.

As mentioned in the previous answer, there are some slight concerns about deploying the model because the fairness assessment showed that the model performs differently across different values of the job variable. Ethically, the problem this could cause is unequal opportunity for people if we under target a certain group because of biases in our model. These disparities were noticed in both validation and testing, which indicates that they are a structural part of the data rather than sampling noise. The groups that would be advantaged are those with a higher predicted probability, becuase they would be more likely to be targeted with the financial opportunity. The

disadvantaged groups would be those that recieve lower predicted probabilities, and thus recieve less access/opportunity to financial opportunities. Again, this is not the biggest deal in the world, but it is an issue that should be continuously monitored if we go through with deployment.

Question 13: Deployment Recommendations and Next Steps: Provide your final recommendation: should this model be deployed, deployed with modifications, or not deployed? If recommending deployment, what specific conditions, monitoring, or constraints would you require? What additional analyses, data collection, or model improvements would you prioritize before or after deployment? Consider operational requirements, maintenance needs, and plans for model updates.

I would suggest that the model be deployed with some modifications/refinements, and consistent monitoring. I do not think it should be thrown away, but also don't think it should be deployed as is. The model has shown strong metrics in discrimination/separation, and showed that it generalizes well to new/unseen data. Also, the model scores well in context based metrics, showing a very low false negative rate (important in marketing) at the selected threshold. All of these factors align and suggest that the model could provide value to the bank. However, there are a few important considerations. The first is the fairness issue we have already noticed. This won't completely derail the value of the model, but it is important to ensure that the model is acting ethically throughout its use, so the issue of bias towards certain groups needs to be monitored.

On top of that, the strong imbalance in the data limits a lot that can be done with predictive analytics. If the bank was able to collect more balanced data, with more variables in it for prediction, the model could be greatly improved for them. If this is the case, and additional data is being collected, I would recommend to try and target under represented job groups in that data collection to start addressing the fairness issue. The last thing I want to discuss in terms of deployment is sustainability. If the model is deployed, it will have to be retrained periodically. I would recommend re-training with modernized data at least twice a year. I think that with effective communication and an understanding of the models limitations, it could be modified and deployed to create value for the bank.

---

8. Save final results

Save your test results for potential future analysis or reporting.

```r
# Create final results object
finalResults <- list(
  testMetrics = testMetrics,
  testPredProbs = testPredProbs,
  testMetricsThres = testMetricsThres,
  selectedLearner = learnerName,
  selectedThreshold = selectedThreshold,
  testROC_AUC = testMetrics$roc_auc,
  testPR_AUC = testMetrics$pr_auc
)

# Add fairness results if calculated
if (exists("equityResults")) {
  finalResults$equityResults <- equityResults
  finalResults$equityTests <- equityTests
}

# Save final results
saveRDS(
  finalResults,
  file = paste0(params$my_path, "finalTestResults.rds")
)

cat("Final results saved successfully!\n")
```

```
## Final results saved successfully!
```

```r
cat("File:", paste0(params$my_path, "finalTestResults.rds"), "\n")
```

```
## File: /share/dsa495601f25/tmgrover/finalTestResults.rds
```

Congratulations! You have completed Milestone 6 - the final testing phase of your predictive modeling project. Make sure you have:

- ☐ **Answered all questions marked with @TODO**
- ☐ **Reviewed test performance against validation baselines**
- ☐ **Assessed fairness and potential biases**
- ☐ **Provided clear deployment recommendations**
- ☐ **Knitted to HTML successfully**
- ☐ **Saved as PDF with correct naming convention:** `DSC495_601_FA25_Milestone6_unityID.pdf`

This completes your predictive modeling project pipeline from data processing through final model testing!